Lab Handout: Autonomous Warehouse Optimization with Multi-Armed Bandits

Course: Reinforcement Learning Fundamentals

Module: Exploration vs. Exploitation Trade-offs

Estimated Time: 75–90 Minutes

1. Problem Specification

In an automated fulfillment center, Autonomous Guided Vehicles (AGVs) are assigned to collect inventory items from 4 distinct warehouse servicing zones (Z_1, Z_2, Z_3, Z_4). Each zone exhibits an unknown underlying probability of having a high-priority dispatch item ready for collection on any given time step.

We model this task as a **Multi-Armed Bandit (MAB)** problem:

Zones (Actions): A = {0, 1, 2, 3}
Zone Success Probabilities: μ = [μ_1, μ_2, μ_3, μ_4] = [0.1, 0.5, 0.8, 0.3]

Reward Model

When an agent selects action A_t = a ∈ A at time step t, the environment samples a binary reward R_t ∈ {0, 1} from a Bernoulli distribution with mean μ_a:

P(R_t = 1 | A_t = a) = μ_a
The maximum expected single-step reward is v* = max_a μ_a = μ_2 = 0.8 (Zone 3, index 2).

Performance Metrics

Agents are evaluated on two complementary criteria over $T = 1000$ steps:

1. **Cumulative Reward (S_T):** The total accumulated dispatches completed over time:

   S_T = Σ R_t (from t=1 to T)
2. **Expected Cumulative Regret (L_T):** The expected cumulative reward lost due to selecting suboptimal zones:

   L_T = Σ (v* - μ_{A_t}) = T · v* - Σ μ_{A_t}

Your objective is to correct buggy implementations of **ε-Greedy** and **Upper Confidence Bound (UCB)** action-selection strategies, implement adaptive **ε-decay schedules**, and analyze their empirical regret growth.

2. Directory & Repository Structure

Your workspace is structured into the following files:

Plaintext

```
None
Bandits-Lab/
├── __init__.py          # Package initialization and class exports
├── environment.py       # 4-Zone Warehouse Environment
(Multi-Armed Bandit)
├── agents.py            # WORKSPACE: Agent implementations to
debug and complete
├── visuals.py           # Rendering functions for status, sweeps,
and regret
├── main.py              # Single-run demonstration and visualizer
├── experiment.py        # Monte Carlo sweep and regret curve
generator
└── checker.py           # Automated diagnostic and grading suite
```

3. Command & API Cheat Sheet

Terminal Execution Commands

Target Task	Command	Description
Run Diagnostic Suite	python checker.py	Runs unit tests against agents.py. Use failed assertions to diagnose logic and math bugs.
Run Demonstration	python main.py	Runs single 1000-step simulations for fixed ε-greedy, decaying ε-greedy, and UCB.
Run Full Experiments	python experiment.py	Executes multi-run Monte Carlo trials ($N=50$) to produce parameter sweeps and regret curves.

Python API Quick Reference

1. Environment Interface (environment.py)

Python

```
None
from environment import WarehouseEnvironment

# Initialize default 4-zone environment
env = WarehouseEnvironment(zone_rates=[0.1, 0.5, 0.8, 0.3])

# Inspect environment attributes
n_arms = env.n_arms          # Integer: 4
rates = env.zone_rates        # Array: [0.1, 0.5, 0.8, 0.3]

# Step the environment
state, reward = env.step(action=2)
# action -> Zone index chosen: 0, 1, 2, or 3
# state  -> Active state array across zones (e.g., [0, 1, 1, 0])
# reward -> 1 if item was collected in selected zone, else 0

# Reset environment state
env.reset()
```

2. Agent Interface (agents.py)

Python

```
None
from agents import EpsilonGreedyAgent, UCBAgent

# Instantiate fixed epsilon-greedy agent
agent = EpsilonGreedyAgent(n_arms=4, epsilon=0.1)

# Instantiate decaying epsilon-greedy agent
agent = EpsilonGreedyAgent(
    n_arms=4,
    epsilon=1.0,
    decay_type='exponential',
    decay_rate=0.995,
    min_epsilon=0.01
```

```
)

# Instantiate UCB agent
agent = UCBAgent(n_arms=4, c=1.0)

# Action selection and update loop
action = agent.select_action()   # Select action (returns integer
0..3)
agent.update(action, reward)     # Update counts, value estimates,
and decay schedules
```

4. Visual Artifacts Reference

Running python experiment.py produces three plot deliverables:

1. **warehouse_bandit_status.png:** Shows true zone probabilities alongside current binary activity states.

2. **hyperparameter_sweep.png:** Displays sensitivity curves mapping hyperparameter settings (ε and c) against average accumulated rewards.

3. **regret_comparison.png:** Plots cumulative expected regret growth (L_T) versus time steps (T).

5. Assigned Tasks (agents.py)

Run python checker.py to identify initial failures, then resolve all issues in agents.py:

Task 1: Action Selection Policy (ε-Greedy)

- **Goal:** Correct the exploration vs. exploitation selection rule inside EpsilonGreedyAgent.select_action().

- **Requirement:** With probability ε, select an action uniformly at random from {0, ..., n_arms - 1}. With probability 1 - ε, select the greedy action argmax_a Q(a).

Task 2: Incremental Sample-Average Updates

- **Goal:** Correct the value update rule inside EpsilonGreedyAgent.update().

- **Requirement:** Update Q(a) using the incremental sample-average formula scaled by the specific action pull count N(a):

  Q_{n+1}(a) = Q_n(a) + (1 / N_n(a)) * [R_n - Q_n(a)]

Task 3: Adaptive Exploration Decay Schedules

- **Goal:** Complete the decay logic inside EpsilonGreedyAgent.update() to reduce ε at each step down to min_epsilon:
  - **Linear Decay (decay_type='linear'):**

    ε_{t+1} = max(ε_min, ε_t - decay_rate)
  - **Exponential Decay (decay_type='exponential'):**

    ε_{t+1} = max(ε_min, ε_t * decay_rate)

Task 4: UCB Action Selection & Boundary Protection

- **Goal:** Implement the complete Upper Confidence Bound selection rule in UCBAgent.select_action():

  A_t = argmax_a [Q_t(a) + c * sqrt(ln t / N_t(a))]
- **Requirement:** Ensure zero-division mathematical errors are prevented when unvisited arms ($N(a) = 0$) exist.

6. Template Starter Code (agents.py)

Copy this code into agents.py as your baseline before debugging:

Python

```
None
import numpy as np

class EpsilonGreedyAgent:
    def __init__(self, n_arms, epsilon=0.1, decay_type=None,
decay_rate=0.01, min_epsilon=0.01):
        self.n_arms = n_arms
        self.epsilon = epsilon
        self.decay_type = decay_type  # Options: None, 'linear',
'exponential'
        self.decay_rate = decay_rate
```

```
        self.min_epsilon = min_epsilon

        self.Q = np.zeros(n_arms)
        self.N = np.zeros(n_arms)
        self.t = 0

    def select_action(self):
        # TODO: Correct epsilon-greedy selection logic
        if np.random.rand() < self.epsilon:
            return np.argmax(self.Q)
        else:
            return np.random.randint(self.n_arms)

    def update(self, action, reward):
        self.N[action] += 1
        self.t += 1

        # TODO: Correct incremental Q-value update rule
        self.Q[action] += (1.0 / self.t) * (reward -
self.Q[action])

        # TODO: Implement epsilon decay schedule if decay_type is
specified
        if self.decay_type == 'linear':
            pass
        elif self.decay_type == 'exponential':
            pass


class UCBAgent:
    def __init__(self, n_arms, c=1.0):
        self.n_arms = n_arms
        self.c = c
        self.Q = np.zeros(n_arms)
        self.N = np.zeros(n_arms)
```

```
    def select_action(self):
        t = np.sum(self.N) + 1

        # TODO: Handle unvisited arms (N[a] == 0) and implement
full UCB formula
        bonus = self.c * (np.log(t) / self.N)
        ucb_values = self.Q + bonus

        return np.argmax(ucb_values)

    def update(self, action, reward):
        self.N[action] += 1
        self.Q[action] += (1.0 / self.N[action]) * (reward -
self.Q[action])
```

7. Submission Checklist & Analytical Questions

Submit a zip file containing your completed agents.py, all generated plot files, and a brief report addressing the following questions:

1. **Automated Validation:** Verify that python checker.py outputs OK.

2. **Experimental Artifacts:** Ensure warehouse_bandit_status.png, hyperparameter_sweep.png, and regret_comparison.png are generated by running python experiment.py.

3. **Analytical Responses:**

    - Why does fixed ε-greedy produce a linear cumulative regret curve O(T), whereas UCB produces a logarithmic regret curve O(log T)?

    - How does adding exponential decay to ε-greedy alter its asymptotic regret growth curve compared to fixed ε?

    - In a dynamic warehouse where zone rates shift over time, why might an agent with logarithmic regret perform worse than one maintaining a small non-zero ε?